

BAB 8

ALGORITMA PREDIKSI MENGGUNAKAN REGRESI

8.1 TUJUAN PERCOBAAN

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep dasar regresi linier dan polinomial dalam konteks pemodelan hubungan antara variabel input dan *output*.
2. Mengimplementasikan algoritma regresi linier dan polinomial untuk memprediksi nilai kontinu menggunakan data multivariat.
3. Mengevaluasi performa model regresi menggunakan metrik evaluasi seperti Mean Squared Error (MSE) dan R-squared (R^2).

8.2 ALAT-ALAT YANG DIGUNAKAN

Pada percobaan ini, akan digunakan berbagai alat dan bahan yang mendukung kegiatan praktikum. Alat-alat tersebut meliputi:

- Daftar alat utama: OS Windows/Linux/macOS, Anaconda, Google Colab
- Pustaka: *pandas*, *numpy*, *matplotlib*, *seaborn*

8.3 DASAR TEORI

8.3.1 REGRESI

Regresi merupakan salah satu algoritma dalam machine learning yang digunakan untuk **melakukan prediksi nilai kontinu** berdasarkan satu atau lebih variabel input (fitur). Tidak seperti algoritma klasifikasi yang memisahkan data ke dalam kelas, regresi bertujuan **mencari hubungan matematis** antara fitur dan target, kemudian memanfaatkan hubungan ini untuk melakukan prediksi.

8.3.1.1 Regresi Linier

Regresi linier mencari model berupa fungsi linier:

$\hat{y}$ adalah prediksi *output*, x_i adalah fitur, dan w_i adalah koefisien (bobot). Tujuan utama adalah mencari bobot-bobot tersebut agar hasil prediksi $\hat{y}$ sedekat mungkin dengan nilai target aktual y .

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

Evaluasi akurasi prediksi dilakukan dengan menghitung **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

8.3.1.2 Regresi Polinomial

Regresi polinomial merupakan perluasan dari regresi linier, dengan menambahkan **fitur berpangkat** (polinomial) agar dapat memodelkan hubungan non-linier:

$$\hat{y} = w_0 + w_1x + w_2x^2 + \dots + w_dx^d$$

Model ini tetap linier terhadap parameter w , namun lebih fleksibel dalam memodelkan data yang tidak memiliki pola linear.

8.3.2 OPTIMASI MODEL REGRESI DENGAN STOCHASTIC GRADIENT DESCENT (SGD)

Untuk memperoleh bobot yang optimal pada regresi linier atau polinomial, digunakan algoritma **Stochastic Gradient Descent (SGD)** yang bertujuan **meminimalkan nilai fungsi kerugian (cost function)**, yaitu MSE. Cara kerja SGD secara umum:

1. Inisialisasi bobot secara acak (biasanya nol).
2. Lakukan iterasi terhadap setiap data:
 1. Hitung prediksi $\hat{y}$.
 2. Hitung error $e = \hat{y} - y$.
 3. **Hitung gradien:** Tentukan arah penyesuaian parameter dengan menghitung turunan/gradien dari fungsi biaya terhadap setiap parameter w_j . Gradien ini mengindikasikan arah kemiringan terbesar fungsi biaya.
 4. **Perbarui parameter:** Ubah setiap w_j sedikit demi sedikit berlawanan dengan arah gradien (arah menurun) dengan langkah sebesar *learning rate* (laju pembelajaran) α . Formula pembaruan sederhana: $w_j = w_j - \theta_j := \theta_j - \alpha \partial \text{MSE} / \partial w_j$. Langkah kecil ini diharapkan menurunkan nilai MSE.
 5. **Ulangi:** Kembali ke langkah 2, hitung error lagi dengan parameter baru, dan ulangi prosesnya. Proses iteratif ini diulang terus hingga perubahan pada fungsi biaya sangat kecil (konvergensi) atau hingga nilai error konvergen atau mencapai jumlah iterasi maksimum.

8.3.3 REGRESI MULTIVARIABEL

Pada kasus nyata, sering kali prediksi sebuah nilai kontinu dipengaruhi oleh lebih dari satu fitur atau variabel bebas. **Regresi linear multivariabel** (disebut juga regresi linear berganda) adalah perluasan regresi linier untuk menangani banyak fitur sebagai input. Model regresi linear multivariabel dengan m fitur $x_1, x_2, \dots, x_m$ dapat dituliskan sebagai berikut:

$$\hat{y} = w_0 + w_1x_1 + \dots + w_nx_n = \mathbf{w}^T \mathbf{x}$$

$\mathbf{x} = [x_1, x_2, \dots, x_m]$ merepresentasikan vektor fitur berdimensi m . Model ini merupakan persamaan hyperplane di ruang berdimensi $m + 1$ (satu dimensi tambahan untuk w_0). Setiap parameter w_j mewakili pengaruh linear dari fitur x_j terhadap prediksi *output*, dengan w_0 sebagai bias (nilai prediksi saat semua $x_j = 0$). Regresi linear multivariabel bekerja dengan prinsip yang sama seperti regresi linear

sederhana: model memprediksi nilai kontinu dengan mengkombinasikan input secara linear, dan parameter w_j ditentukan melalui pelatihan dengan meminimalkan fungsi biaya MSE pada data latih. Metode optimasi yang dijelaskan sebelumnya (seperti *gradient descent*/SGD) secara langsung dapat diaplikasikan pada kasus multivariabel, karena rumus turunan MSE dapat diperumum untuk setiap w_j . Demikian pula, fungsi error yang digunakan sama dengan kasus univariat (MSE) dan pendekatan pencarian parameter (misalnya dengan *least squares* atau *gradient descent*) tetap berlaku pada regresi multivariabel.

8.4 IMPLEMENTASI REGRESI LINIER DAN REGRESI POLINOMIAL

8.4.1 DATASET SINTESIS

Pada praktikum ini, Anda akan mengimplementasikan regresi linier dan regresi polinomial tanpa menggunakan library machine learning seperti scikit-learn.

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 np.random.seed(42)
6 X = 2 * np.random.rand(100, 1)
7 y = 4 + 3 * X + np.random.randn(100, 1)
8
9 plt.scatter(X, y)
10 plt.xlabel("X")
11 plt.ylabel("y")
12 plt.title("Data untuk Regresi Linier")
13 plt.show()
```

8.4.2 REGRESI LINIER

Tujuan: Mengimplementasikan regresi linier sederhana tanpa library ML.

Model regresi linier:

$$\hat{y} = w_0 + w_1x$$

Gunakan **gradient descent** untuk memperbarui parameter w_0 dan w_1 .
Fungsi loss yang digunakan: **Mean Squared Error (MSE)**

```
1
2 w0 = 0
3 w1 = 0
4 alpha = 0.1
5 epochs = 1000
6 m = len(X)
7
8 # Gradient Descent
9 for epoch in range(epochs):
10     y_pred = w0 + w1 * X
11     error = y_pred - y
12     grad0 = (2/m) * np.sum(error)
```



```
13     grad1 = (2/m) * np.sum(error * X)
14
15     w0 -= alpha * grad0
16     w1 -= alpha * grad1
17
18     print(f"w0: {w0}, w1: {w1}")
19
20     plt.scatter(X, y, label="Data")
21     plt.plot(X, w0 + w1 * X, color='red', label="Regresi Linier")
22     plt.legend()
23     plt.show()
24     plt.scatter(X, y, label="Data")
```

8.4.3 REGRESI POLINOMIAL

Tujuan: Mengimplementasikan regresi polinomial derajat 2.

Model polinomial:

$$\hat{y} = w_0 + w_1x + w_2x^2$$

```
25
26 # Buat fitur polinomial
27 X_poly = np.hstack((np.ones((m, 1)), X, X**2)) # tambahkan bias term
28 w = np.zeros((X_poly.shape[1], 1))
29
30 # Gradient Descent
31 for epoch in range(epochs):
32     y_pred = X_poly @ w
33     error = y_pred - y
34     gradients = (2/m) * X_poly.T @ error
35     w -= alpha * gradients
36
37 print("w:", w.ravel())
38
39 x_plot = np.linspace(0, 2, 100).reshape(-1, 1)
40 x_poly_plot = np.hstack((np.ones_like(x_plot), x_plot, x_plot**2))
41 y_plot = x_poly_plot @ w
42
43 plt.scatter(X, y, label="Data")
44 plt.plot(x_plot, y_plot, color='green', label="Regresi Polinomial (derajat 2)")
45 plt.legend()
46 plt.show()
```

instruksi, soal coding, dan soal analisis:

8.4.4 TUGAS PRAKTIKUM: REGRESI LINIER DAN POLINOMIAL

1. Instruksi Umum:

1. Kerjakan seluruh tugas di bawah ini menggunakan **NumPy**, tanpa menggunakan library machine learning seperti scikit-learn.
2. Simpan jawaban Anda dalam file .ipynb dan sertakan penjelasan singkat (komentar) di setiap langkah.
3. Gunakan **visualisasi** untuk mendukung hasil analisis Anda.

2. Tugas A: Regresi Linier

Buat *dataset* sintetis baru dengan rumus:

$$y = 7 + 2.5x + \epsilon, \text{ dengan } \epsilon \sim N(0,1)$$

Gunakan 200 data acak dari rentang $x \in [0,5]$. Implementasikan algoritma **regresi linier** dari awal menggunakan **gradient descent**. Cetak nilai akhir dari parameter w_0 dan w_1 .

1. Hitung dan tampilkan:
 - o Nilai **Mean Squared Error (MSE)**
 - o Plot garis regresi di atas data
2. Ubah nilai **learning rate** (α) menjadi 0.001 dan 1.0. Amati dan jelaskan perbedaan konvergensinya.

3. Tugas B: Regresi Polinomial

Buat *dataset* sintetis baru dengan pola kuadratik:

$$y = 5 - 1.2x + 0.9x^2 + \epsilon$$

Gunakan 150 data acak dari $x \in [-2,2]$

1. Buat regresi **polinomial derajat 2** dari awal. Implementasikan gradient descent dan tampilkan parameter w_0, w_1, w_2
2. Bandingkan MSE antara:
 - o Regresi linier (fit hanya $w_0 + w_1x$)
 - o Regresi polinomial (fit hingga $w_2 x^2$)
3. Visualisasikan kedua model regresi pada grafik yang sama dan diskusikan model mana yang lebih baik? Apa indikatornya?

4. Tugas C: Eksplorasi dan Analisis

1. Tambahkan derajat polinomial menjadi 5 (quintik). Apakah model menjadi lebih baik atau justru overfitting?
2. Buat fungsi `predict(x, w)` yang menerima data baru dan bobot hasil training, lalu mengembalikan hasil prediksi.
3. Tulis kesimpulan Anda:
 - o Kapan kita perlu menggunakan regresi linier?
 - o Kapan regresi polinomial lebih cocok?
 - o Apa dampak jumlah epoch dan learning rate terhadap performa training?

8.5 KESIMPULAN

Pada praktikum ini, telah dilakukan implementasi regresi linier dan regresi polinomial dari awal (*from scratch*) tanpa menggunakan library machine learning seperti `scikit-learn`. Secara

keseluruhan, praktikum ini memberikan pemahaman mendalam tentang konsep dasar regresi, algoritma optimasi, serta evaluasi performa model prediktif berbasis MSE.

Setelah melakukan praktikum ini mahasiswa diwajibkan untuk membuat kesimpulan.

Kesimpulan dari percobaan ini merupakan bagian penting untuk merangkum temuan utama dan relevansinya dengan tujuan percobaan. Mahasiswa harus mampu menyimpulkan:

- Apakah tujuan percobaan telah tercapai.
- Hubungan antara hasil percobaan dan dasar teori.
- Faktor-faktor yang memengaruhi hasil percobaan, termasuk kemungkinan kesalahan yang terjadi.

Kesimpulan harus disusun secara singkat, jelas, dan berdasarkan data yang diperoleh